DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2026

Input/Output

- Introduction
- I/O Types
  Programmed I/O
  Interrupted-Driven
  DMA
  Direct Cache Access
  I/O Channels
- External Interfaces

Overview: PCIe

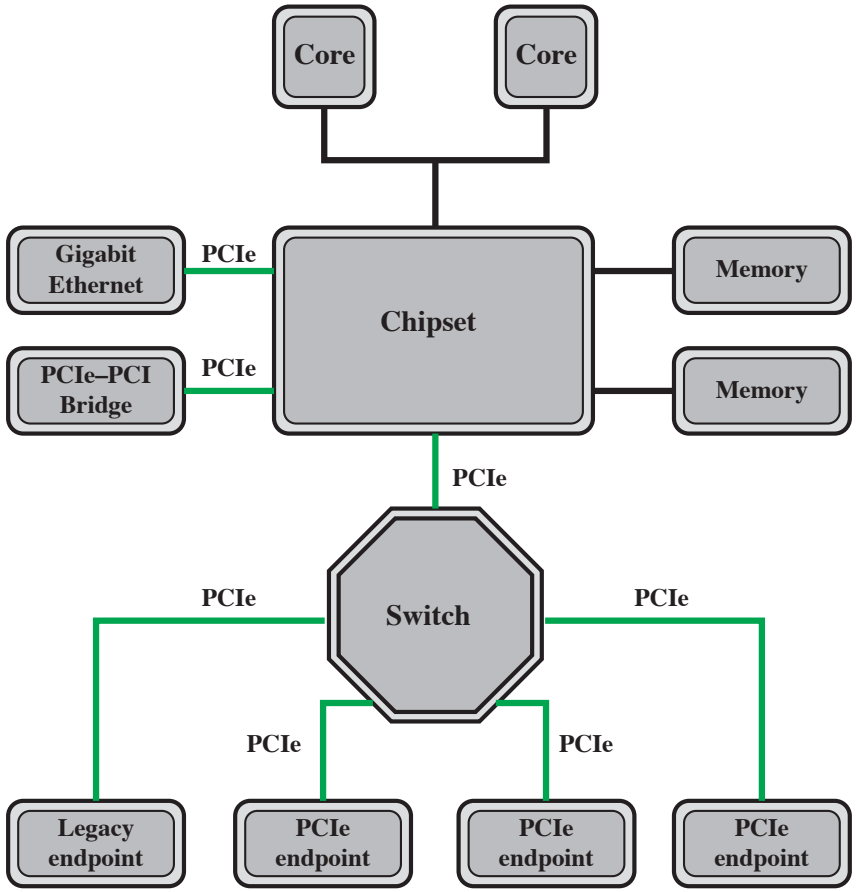
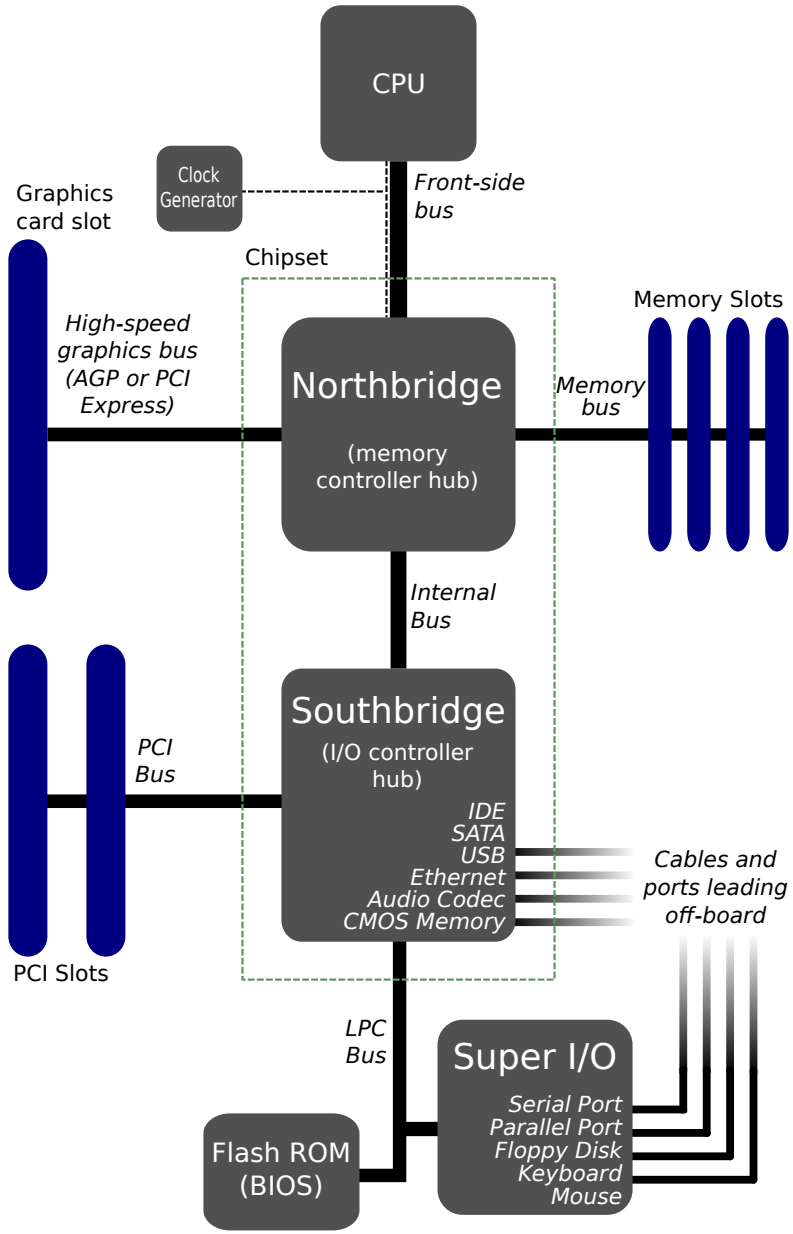


Figure 3.21 Typical Configuration Using PCIe

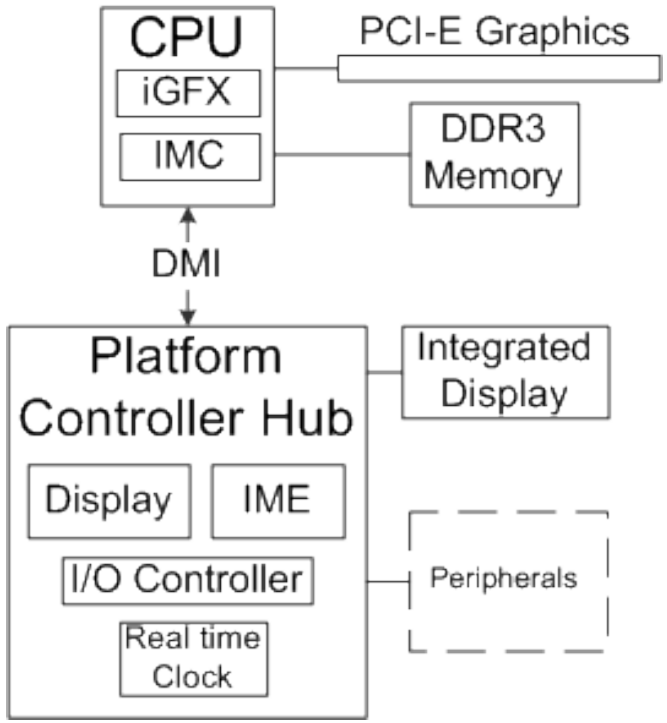
Overview: Typical Bus-Based Organisation



[https://en.wikipedia.org/wiki/Northbridge_(computing)]

Overview: Modern Organization (Intel)

The shrinking transistor size made it possible to integrate the northbridge in the CPU from ∼2010.



Further shrinking has made it possible to integrate some of the southbridge/PCH as well.
This is called a system-on-chip (SoC) design.

[https://en.wikipedia.org/wiki/Platform_Controller_Hub]

General Model for I/O Modules

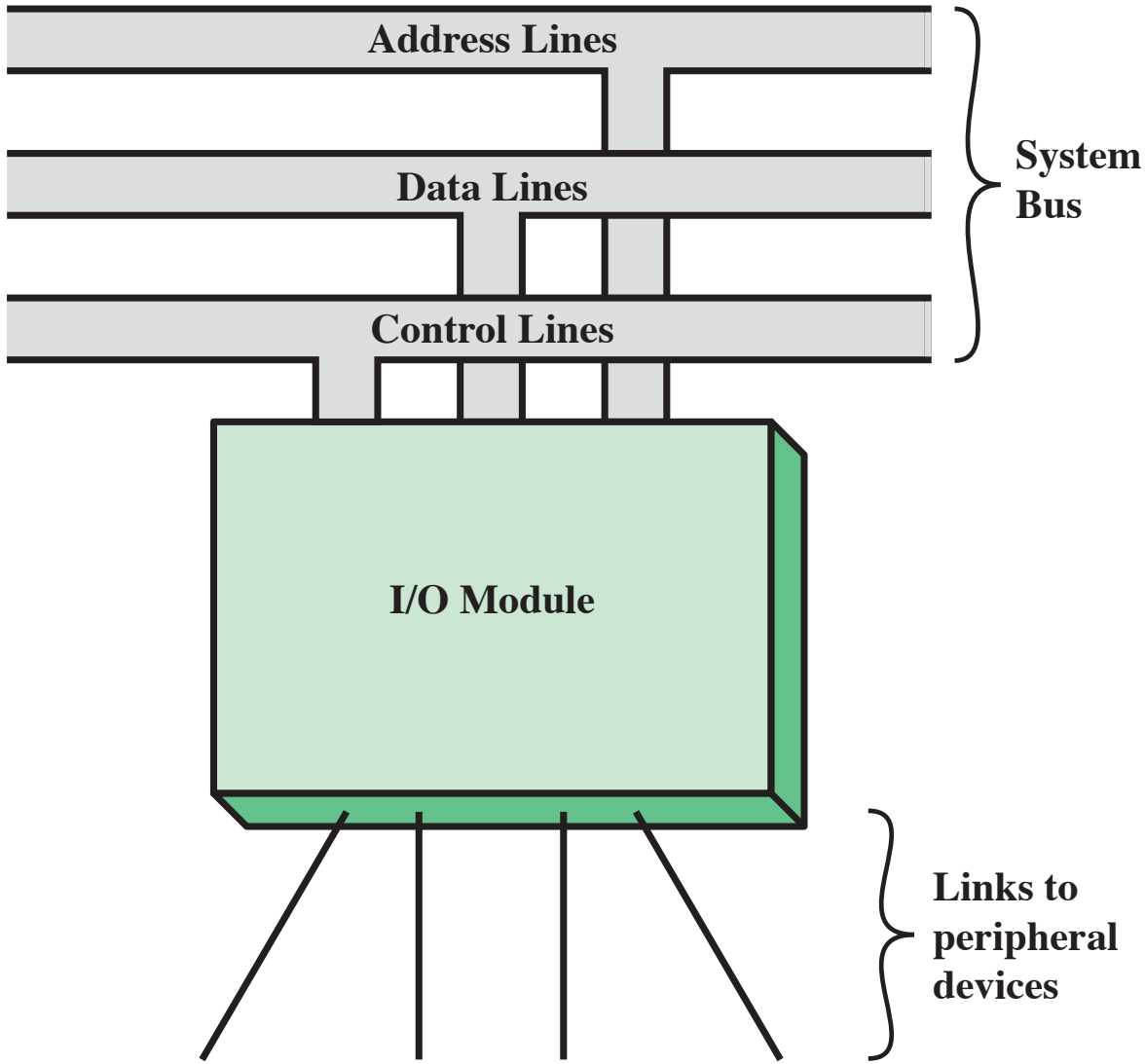


Figure 8.1 Generic Model of an I/O Module

External Device (Peripheral)

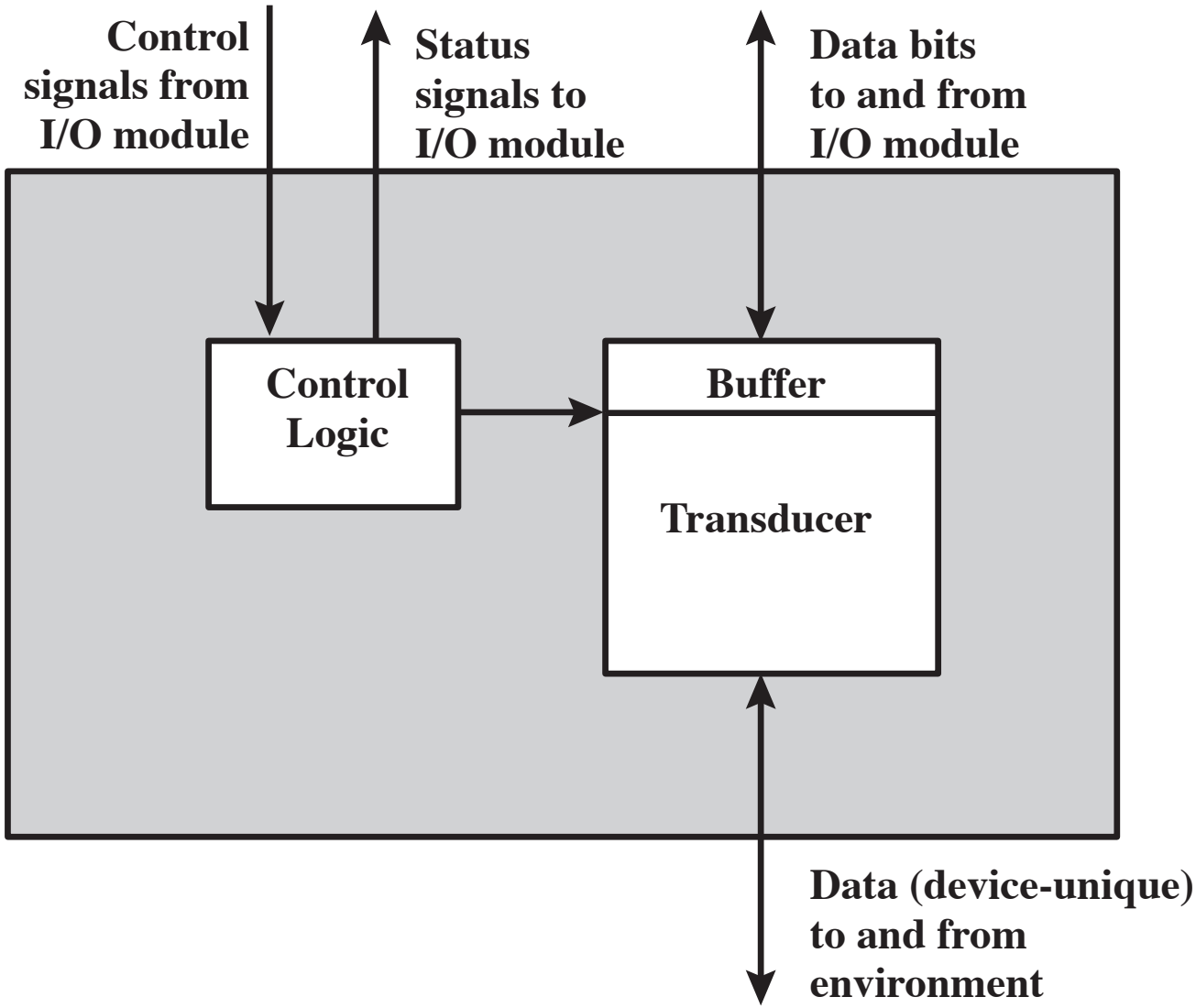
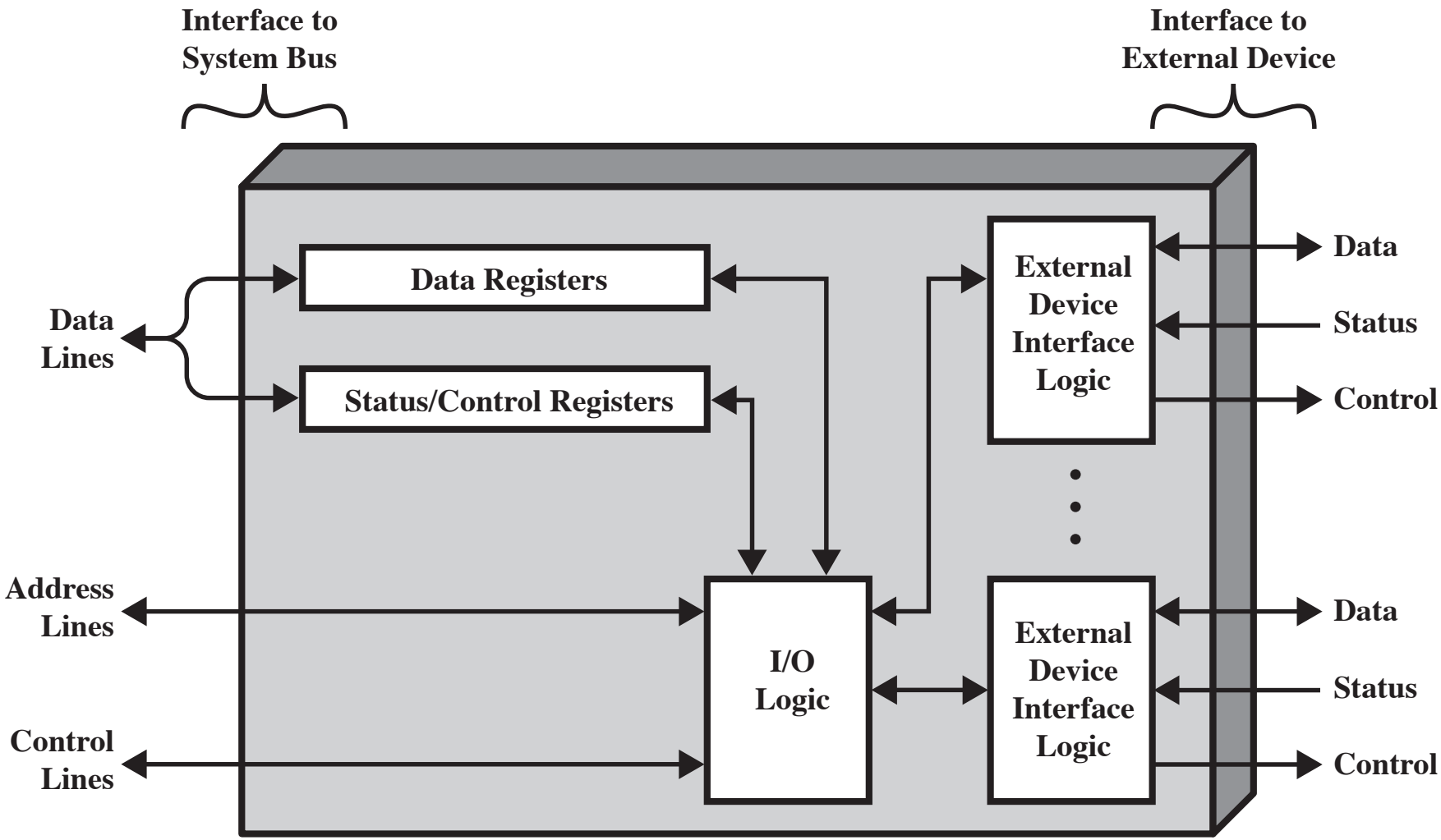


Figure 8.2 Block Diagram of an External Device

I/O Module


Interface to System Bus
Interface to External Device
Data Registers
Status/Control Registers
Data Lines
Address Lines
Control Lines
I/O Logic
External Device Interface Logic
External Device Interface Logic
Data
Status
Control
Data
Status
Control

Why Using I/O Modules?

- Allows the CPU to communicate with a wide range of devices in a simple-minded way.
- The I/O module may hide the details of timing, formats, and the electromechanics of an external device.
- The CPU can function in terms of simple read and write commands.
- In its simplest form, the I/O module may still leave much of the work of controlling a device (e.g., rewind a tape) visible to the CPU.

Duties of an I/O Module

- Control and timing: coordinate the traffic between internal resources and external devices.
- CPU communication: command decoding, data, status reporting, address recognition.
- Device communication: commands, status information, data.
- Data buffering: perform buffering to balance the speed of the device and memory.
- Error detection: detect, correct, and report errors.

Naming Confusion

An I/O module is often categorized as one of

- I/O channel or I/O processor:
  - Takes on most of the detailed processing burden.
  - Presents a high-level interface to the CPU.
- I/O controller or Device controller
  - Relatively primitive.
  - Requires detailed control by the CPU.

I/O controllers are commonly seen in PCs, while I/O channels are used in mainframes.

- Introduction

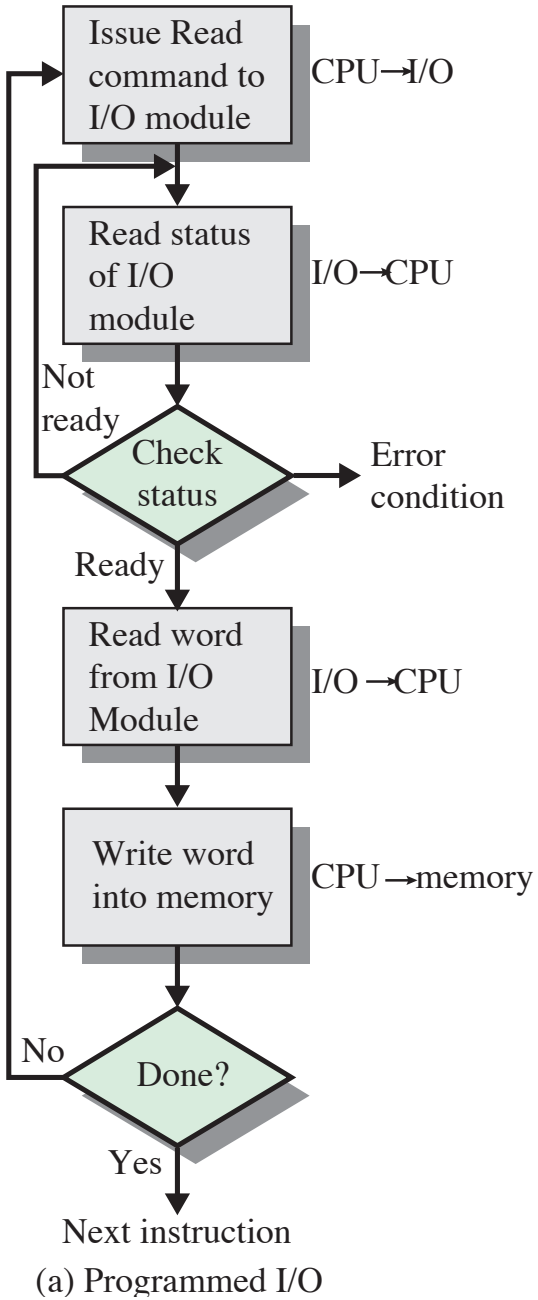
- I/O Types
  Programmed I/O
  Interrupted-Driven
  DMA
  Direct Cache Access
  I/O Channels

- External Interfaces

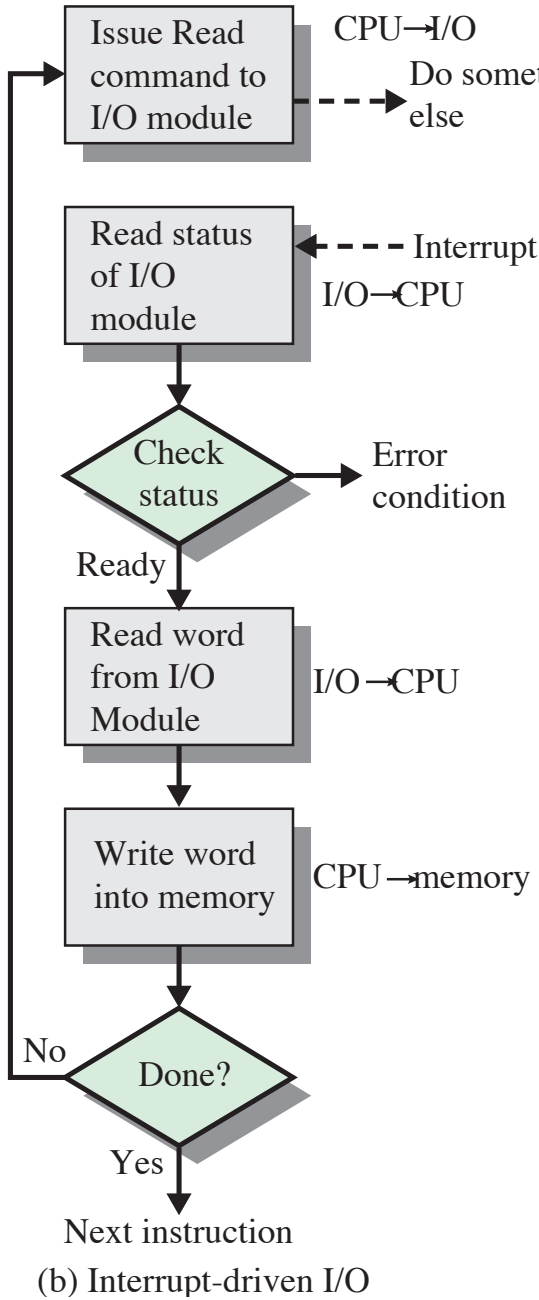
Different Types of I/O

- Programmed I/O:
  - Data is exchanged between the CPU and the I/O module.
  - The CPU has direct control of the I/O operation.
  - The CPU must check if an I/O operation is complete.
  - Wasteful, if the CPU is faster than the I/O device.
- Interrupt-driven I/O:
  - Data is exchanged between the CPU and the I/O module.
  - The CPU has direct control of the I/O operation.
  - The CPU can perform other tasks, and get notified of I/O completion by an interrupt.
- Direct memory access (DMA):
  - The CPU initiates a higher-level operation and goes off to do something else.
  - The I/O module and main memory exchange data directly without processor involvement.
  - The CPU gets notified by an interrupt when the higher-level operation is completed.

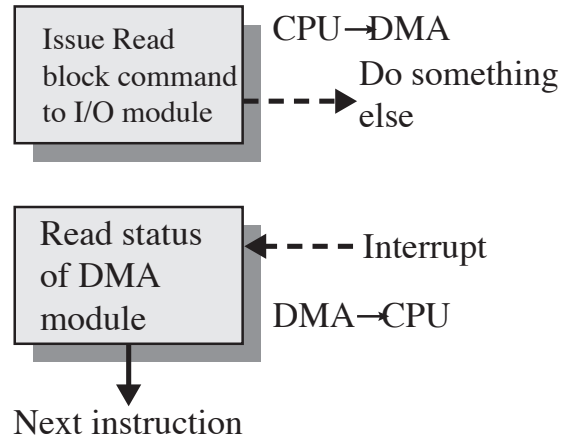
Techniques for Input of a Block of Data



(a) Programmed I/O



(b) Interrupt-driven I/O



(c) Direct memory access

- Introduction
- I/O Types
  - Programmed I/O
  - Interrupted-Driven
  - DMA
  - Direct Cache Access
  - I/O Channels
- External Interfaces

I/O Commands

- Control: used to activate a peripheral device and tell it what to do.
- Test: used to test various status conditions of an I/O module and its peripherals.
- Read: causes the I/O module to obtain an item of data from the peripheral, and place it in an internal buffer.
- Write: causes the I/O module to take an item of data from the data bus and subsequently transmit that data item to the peripheral.

I/O Instructions with Programmed I/O

- Close correspondence between the I/O instruction and the device.
- The instruction depends on the way in which external devices are addressed.
- Typically there are several devices:
  - Each device has an identifier or address.
  - The command contains the address of the desired device.
  - Thus each I/O module must interpret the address lines to determine if the command is for it self.
- Sharing the bus with CPU, main memory, and I/O.
  - Memory mapped I/O:
    - Devices and memory share an address space.
    - I/O looks just like memory read/write.
    - No special instructions for I/O.
    - Large selection of memory access instructions available.
  - Isolated I/O:
    - Separate address spaces.
    - Need lines to select I/O or memory.
    - Special instructions for I/O.

- Introduction
- I/O Types
  - Programmed I/O
  - Interrupted-Driven
  - DMA
  - Direct Cache Access
  - I/O Channels
- External Interfaces

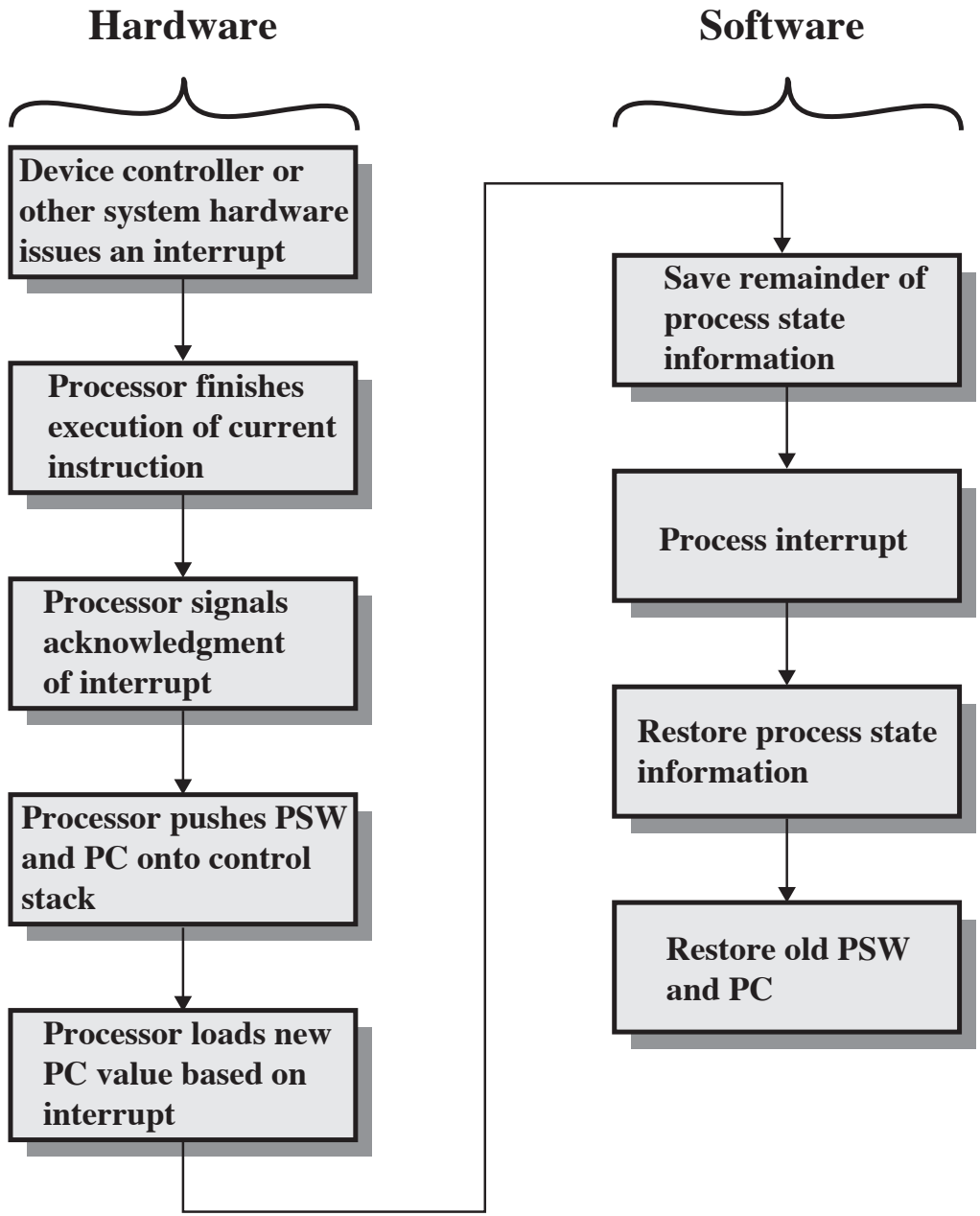
Interrupt-Driven I/O

- The problem with programmed I/O is that the CPU has to wait and check on the I/O module all the time.
- An alternative is for the processor to issue an I/O command to a module, and then go on to do some other useful work.
- The I/O module will then interrupt the CPU to request service when it is ready to exchange data with the CPU.
- The CPU executes the data transfer and resumes its former processing.

Two fundamental implementation issues:

- There will be multiple I/O modules, so how does the CPU determine which device issued an interrupt?
- If multiple interrupts have occurred, how does the CPU decide which one to process first?
  (Recall: sequential, nested, prioritized)

Simple Interrupt Processing



Device Identification

- Multiple interrupt lines into the CPU
  - Most straight-forward approach to the problem.
  - But we can only have so many lines, so some devices must share.
- Software polling
  - One interrupt line, when raised the CPU branches to an interrupt-service routine. The routine will poll each I/O module to determine which of them cause the interrupt.
  - Time consuming.
- Daisy chaining (hardware poll, vectored)
  - The interrupt acknowledge line is daisy chained through the modules.
  - Modules are serviced in the order of the chain.
  - Each module has some unique identifier, its vector, which it puts on the bus when its interrupt gets acknowledged-
  - The identifier is used by the CPU to branch to the correct interrupt handler directly.
- Bus arbitration (vectored)
  - An I/O module only issues an interrupt when it has gained control of the bus.
  - Otherwise as daisy chaining.

Example of Daisy-Chain Configuration

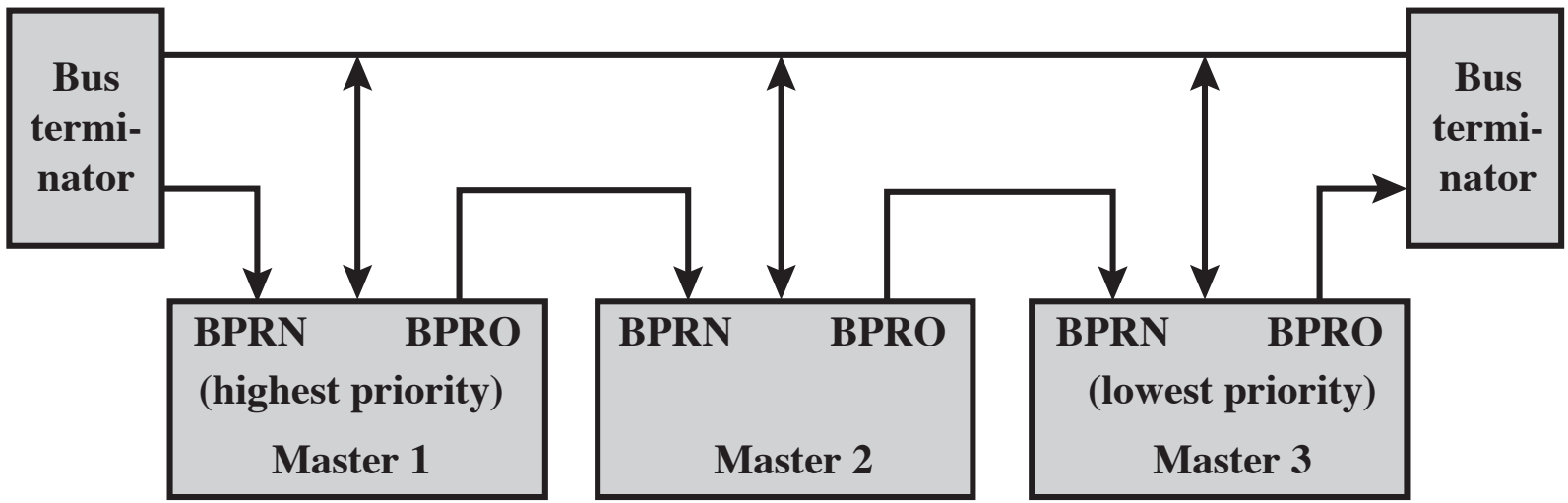
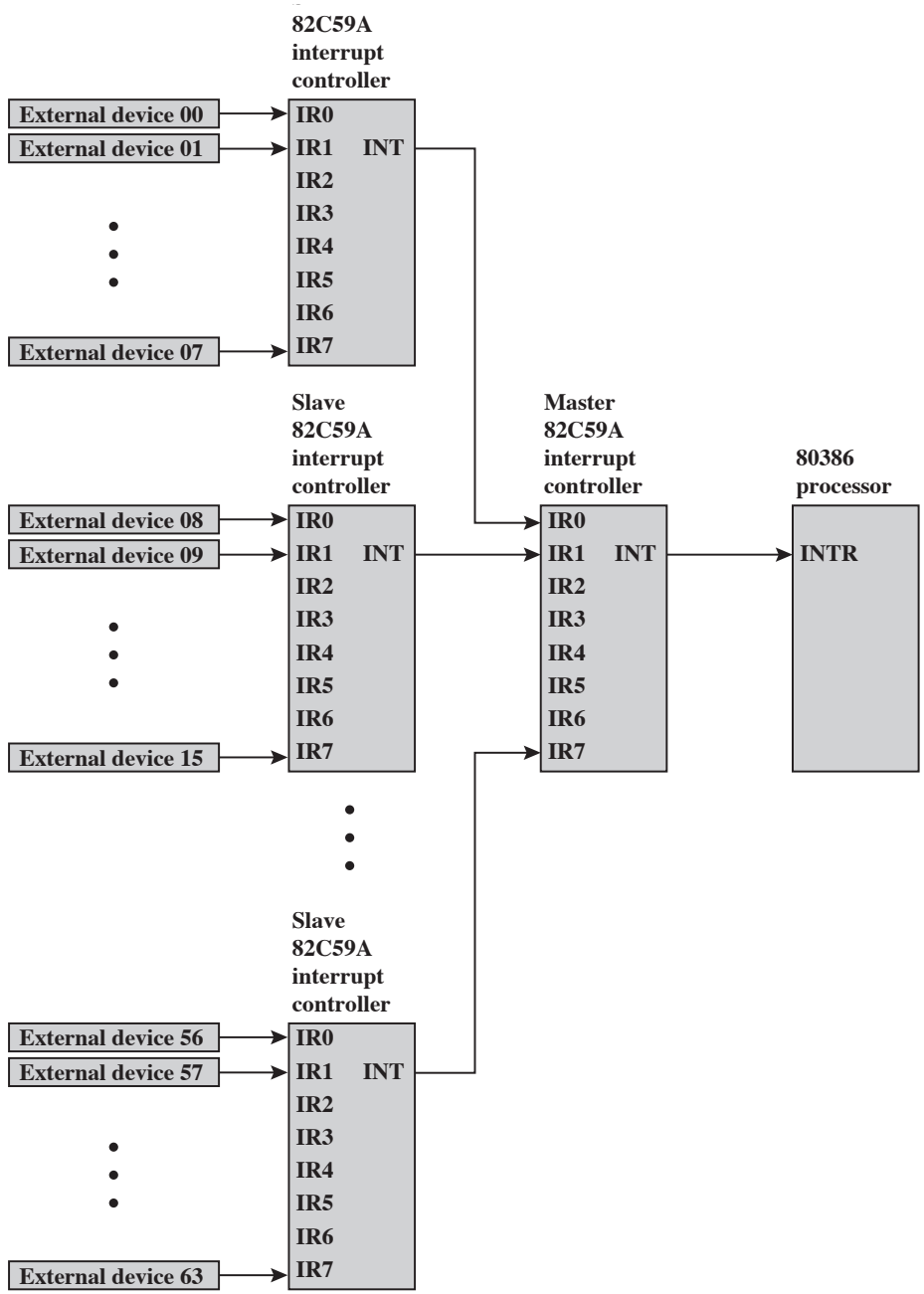


Figure 3.26 Multibus I Distributed Arbitration

Intel 8259 PIC (Programmable Interrupt Controller)

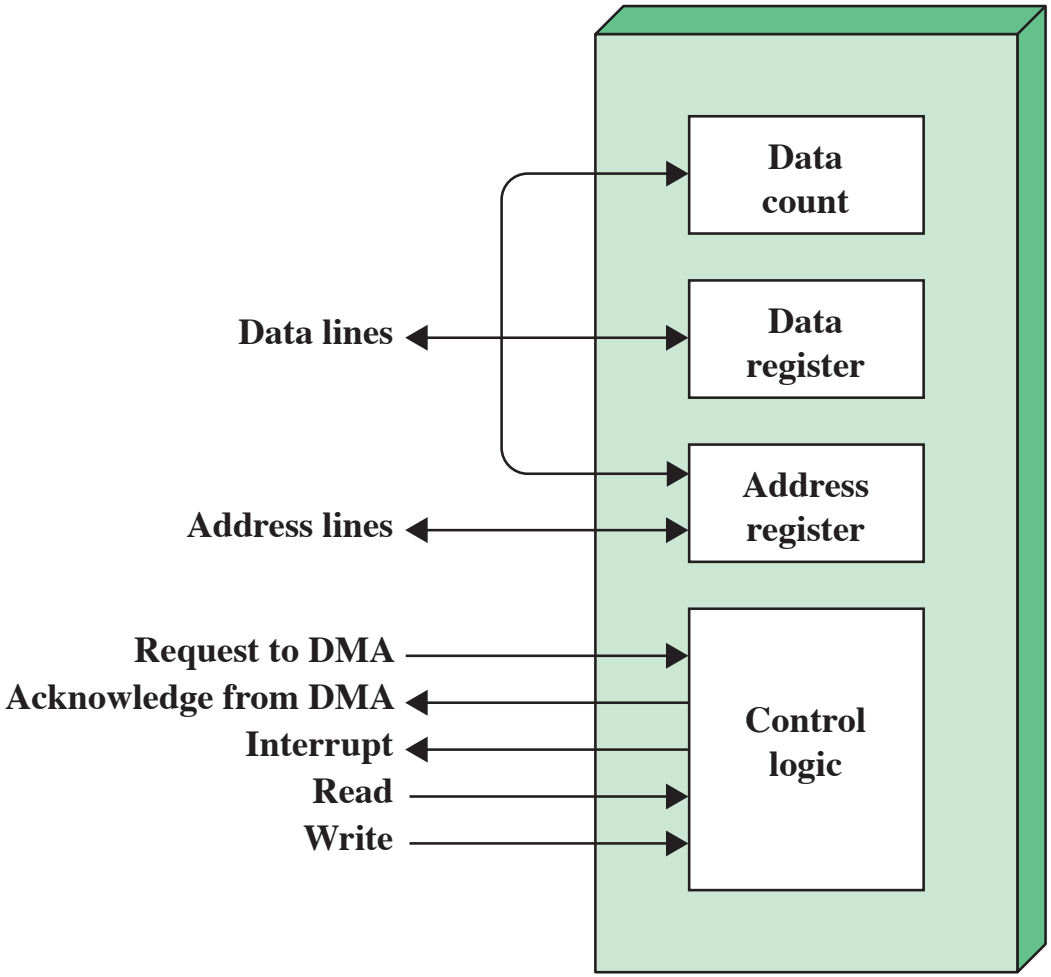


Drawbacks

- Programmed I/O
  - With simple programmed I/O, the CPU is dedicated to the task of I/O and can move data at a rather high rate,
  - but at the cost of doing nothing else. Most of the time the CPU idles while waiting for I/O (busy waiting).
- Interrupt-driven I/O
  - The CPU gets freed up for other tasks, but at the potential expense of a lower I/O transfer rate.
  - In reality the overall performance is better.
  - Though,
    - It is more complicated to coordinate.
    - The CPU is still involved in every data transfer.
    - I/O is limited by how fast the CPU can handle interrupts.

- Introduction
- I/O Types
  - Programmed I/O
  - Interrupted-Driven
  - DMA
  - Direct Cache Access
  - I/O Channels
- External Interfaces

Direct Memory Access (DMA)

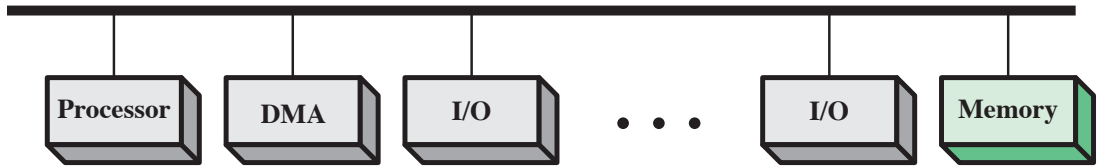


- Requires an additional module.
- After requesting a DMA transaction, the CPU completely delegates the I/O transfer to the DMA module.

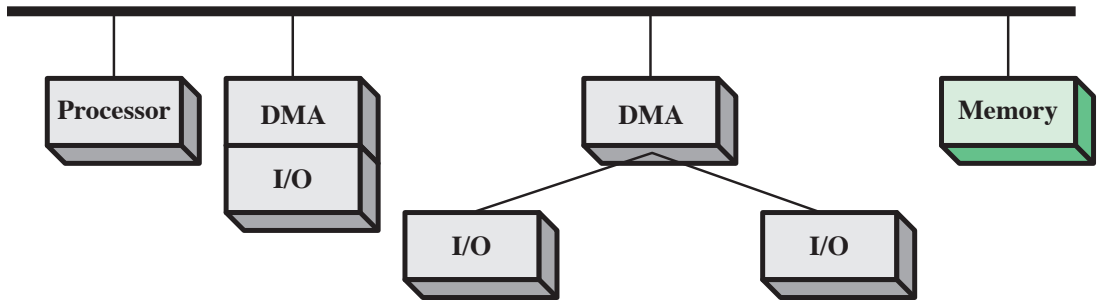
DMA Operation in Detail

- The CPU provides the DMA with the following information:
  - Type: read or write.
  - Address of I/O device.
  - Start of memory location to read/write.
  - Number of words to be read or written.
- The DMA module stores the information in internal registers.
- When the transfer is complete, the DMA module sends an interrupt to the CPU.
- The CPU is thus only involved in the beginning and end of the overall transfer.

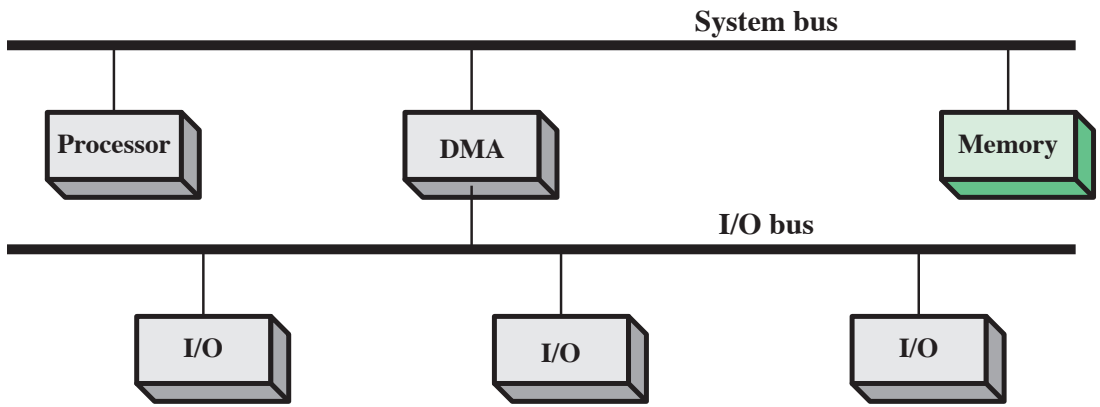
Alternative DMA Configurations



(a) Single-bus, detached DMA



(b) Single-bus, Integrated DMA-I/O



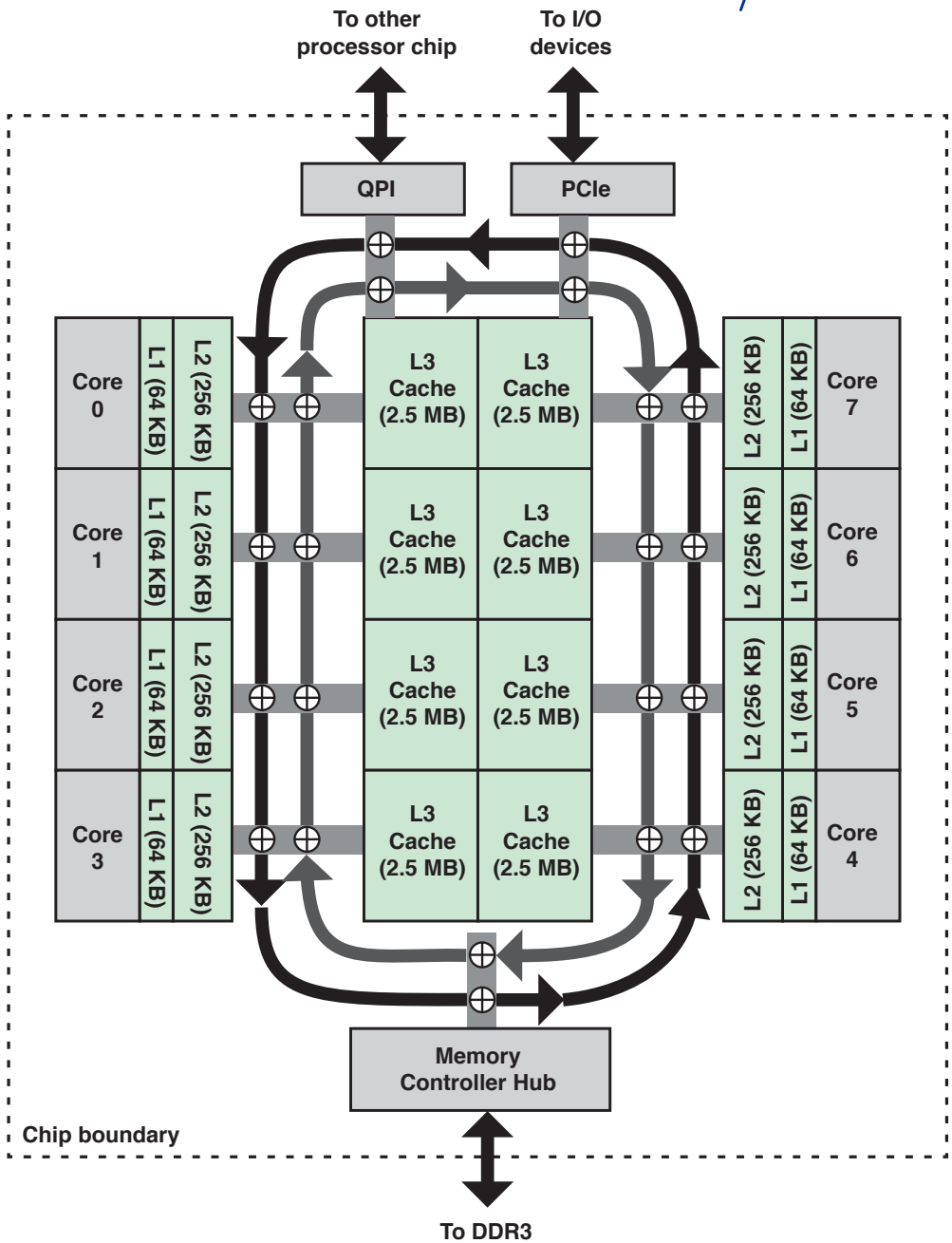
(c) I/O bus

- Introduction
- I/O Types
  - Programmed I/O
  - Interrupted-Driven
  - DMA
  - Direct Cache Access
  - I/O Channels
- External Interfaces

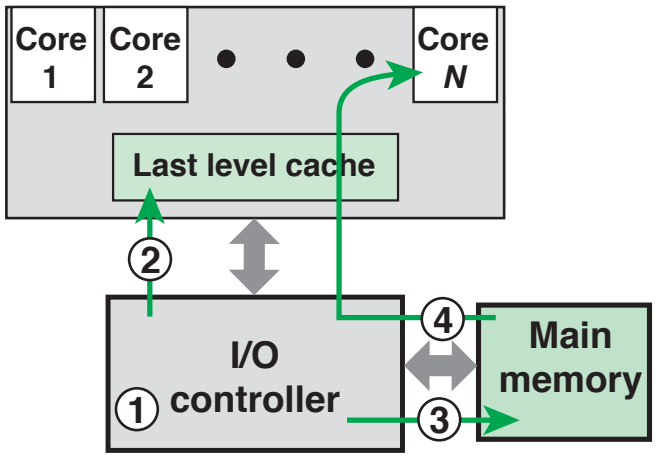
Direct Cache Access (DCA)

- DMA does not scale well with dramatically increased network I/O rates.
- Primarily due to widespread deployment of 10 Gbps and 100 Gbps Ethernet switches.
- But also WiFI traffic getting into the gigabit range.
- Traditionally DMA transfers between main memory and I/O modules,
- but it may be allowed to read from the last-level cache as well.
- With DCA it can also write into the last-level cache from the I/O modules.
- DDIO: Intel Direct Data I/O.

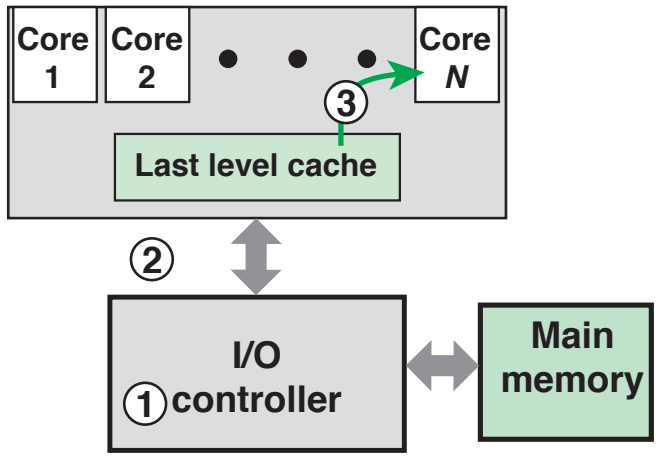
Example System: Intel Xeon E5-2600/4600



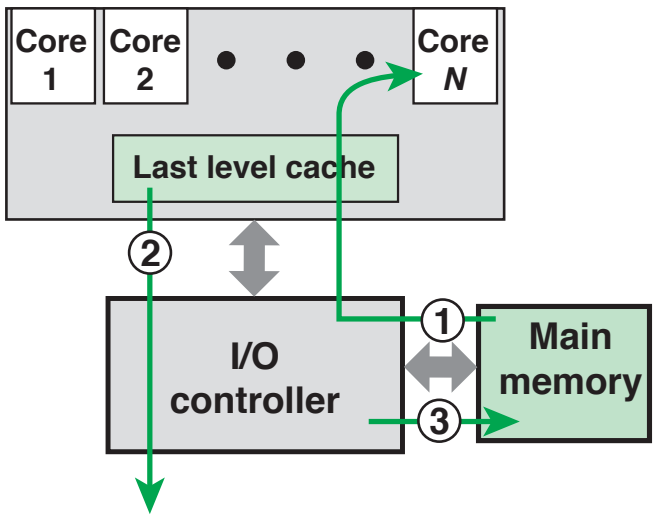
Comparison of DMA and DDIO



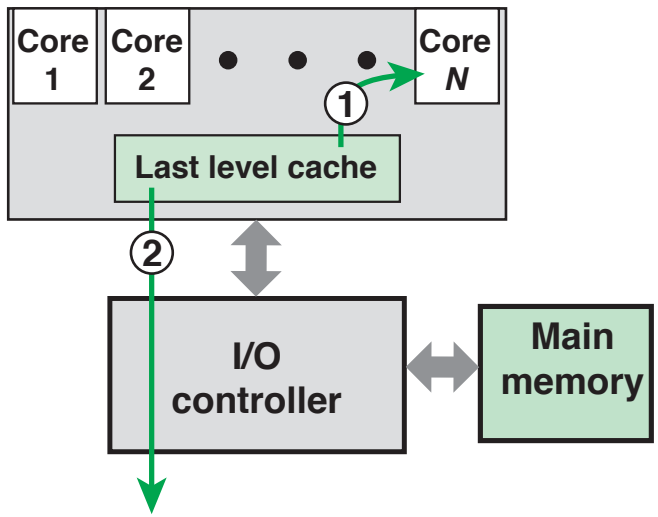
(a) Normal DMA transfer to memory



(b) DDIO transfer to cache



(c) Normal DMA transfer to I/O



(d) DDIO transfer to I/O

Direct Transfers

- DMA enables direct transfer between IO modules and main memory.
- DCA enables direct transfer between IO and the CPU.
- The next step: direct transfer from IO to GPU
  E.g. search for “GPUDirect Storage”.

- Introduction
- I/O Types
  - Programmed I/O
  - Interrupted-Driven
  - DMA
  - Direct Cache Access
  - I/O Channels
- External Interfaces

Evolution of the I/O Function

1. The CPU directly controls a peripheral device.
2. An I/O module is added, the CPU still uses programmed I/O, though it benefits from the abstraction.
3. The same, but with interrupts.
4. DMA
5. The I/O module becomes a specialized processor that can execute special I/O programs, an I/O Channel.
   - The CPU only gets interrupted when the I/O program is done.
   - The CPU no longer needs to execute I/O instructions.
6. The I/O module has local memory and becomes a specialized computer within the system, an I/O Processor.
   - It can perform some types of data processing tasks without the CPU.

- Introduction
- I/O Types
  - Programmed I/O
  - Interrupted-Driven
  - DMA
  - Direct Cache Access
  - I/O Channels
- External Interfaces

Universal Serial Bus (USB)

- The computer acts as the host.
- Up to 127 devices can connect to the host (directly or by hubs).
- Maximum transfer rates:

Name or Version	Year	Speed ($\frac{\text{Mbit}}{\text{s}}$)
USB 1.0	1996	1.5
USB 1.0	1998	12
USB 2.0	2000	480
USB 3.0	2008	5000
USB 3.1	2013	10 000
USB 3.2	2017	20 000
USB4	2019	40 000
USB4 2.0	2022	120 000

- USB naming is a complete mess, and has changed many times.
- USB cables uses twisted pairs of wires with differential signalling.
- Additional wires in the cable provides power to the connected device.

Other

- FireWire Serial Bus: developed as alternative to SCSI, for smaller systems (e.g., PCs, workstations, servers).
- SCSI: Small Computer System Interface. Has lost popularity to USB and FireWire, but still popular on enterprise systems.
- Thunderbolt
  - A recent and fastest general-purpose connection technology.
  - Developed by Intel in cooperation with Apple.
  - USB 4 is based on it.
- PCIe: (already discussed in previous lectures)
  Used for most internal IO connections today.
- SATA: Serial Advanced Technology Attachment.
  Interface for some storage devices.
- Ethernet: the predominant wired network technology.
- WiFi (IEEE 802.11): the predominant wireless network technology.

InfiniBand

- Recent I/O specification aimed at the high-end server market.
- The first version was released in early 2001.
- A standard describes an architecture and provides specification for data flow among processors and intelligent I/O devices.
- Enables servers, remote storage, and other network devices to be attached in a central fabric of switches and links.
- The switch-based architecture can connect up to 64 000 servers, storage systems, and network devices.
- For example used in supercomputers.
- Example: Abacus 2.0 at SDU uses InfiniBand FDR (56 Gbps) in a 3D torus topology.

[https://escience.sdu.dk/index.php/hpc/]